Tanveer

SPRING DATA JPA WITH MYSQL RELATION

Step 1: Application Properties Configuration

```
spring.application.name=jpamysql
server.port=88

# JDBC Setup
spring.datasource.url=jdbc:mysql://localhost:3306/testdb?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.jdbc.Driver

# JPA / Hibernate Setup
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Step 2: Database Schema & Mock Injections

```
CREATE DATABASE IF NOT EXISTS testdb;
USE testdb;

CREATE TABLE IF NOT EXISTS employee (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  department VARCHAR(100)
);

INSERT INTO employee (name, department) VALUES
('Ramesh','IT'),
```

```
('Suresh','CSE'),
('Mahesh','ECE');
```

📷 SCREENSHOT 1: MySQL Server Console View Matrix Output

```
mysql> USE testdb;
Database changed
mysql> SELECT * FROM employee;
+----+---------+--------------------+
| id | name | department |
+----+---------+--------------------+
| 1 | Ramesh | IT |
| 2 | Suresh | CSE |
| 3 | Mahesh | ECE |
+----+---------+--------------------+
3 rows in set (0.00 sec)
```

Step 3: Java Domain Entity Implementation

```
package com.example.demo;

import jakarta.persistence.*;

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String name;
    private String department;

    public Employee() {}
    public Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department =
```

```
department; }
}
```

Step 4: Spring Data JPA Repository Mappings

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Integer>
{ }
```

Step 5: Corporate Dual-Routing Controller Engine

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api")
class EmployeeRestController {
    @Autowired
    private EmployeeRepository repo;

    @GetMapping("/employees")
    public List<Employee> getAll() { return repo.findAll(); }
}

@Controller
class EmployeeViewController {
    @Autowired
    private EmployeeRepository repo;

    @GetMapping("/employees")
    public String employeesPage(Model model) {
        model.addAttribute("employees", repo.findAll());
        return "employees";
```

```
    }
}
```

Step 6: Thymeleaf Layout Target Front-End Template

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <title>Employees</title>
</head>
<body>
  <h2>Employee List</h2>
  <table>
    <thead>
      <tr><th>ID</th><th>Name</th><th>Department</th></tr>
    </thead>
    <tbody>
      <tr th:each="e : ${employees}">
        <td th:text="${e.id}">1</td>
        <td th:text="${e.name}">Name</td>
        <td th:text="${e.department}">Dept</td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

Step 7: Spring Tool Suite Execution Output

📷 SCREENSHOT 2: Spring Tool Suite (STS) System Application Console Runtime Boot Stream

```
2026-11-10T19:51:02.124Z INFO 14832 --- [main] c.e.demo.JpamysqlApplication : Starting JpamysqlApplication v0.0.1
2026-11-10T19:51:04.532Z INFO 14832 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 88 (http)
2026-11-10T19:51:05.101Z INFO 14832 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform
2026-11-10T19:51:05.882Z INFO 14832 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 88 (http) with context path '/'
```

```
2026-11-10T19:51:05.895Z INFO 14832 --- [main] c.e.demo.JpamysqlApplication :
Started JpamysqlApplication in 4.12 seconds
```

Step 8: Front-End Web-Page Endpoint Rendering Output

Web Browser - Address: http://localhost:88/employees

Employee List

ID	Name	Department
1	Ramesh	IT
2	Suresh	CSE
3	Mahesh	ECE

Step 9: RESTful API JSON Matrix Response Payload

Web Browser (API View) - Address: http://localhost:88/api/employees


```
[
  {
    "id": 1,
    "name":
    "Ramesh",
  },
  {
    "id": 2,
    "name": "Suresh",
    "department":
  },
  {
    "id": 3,
    "name": "Mahesh",
    "department":
```

```
  }
]
```